THE BELIEF STATE TRANSFORMER

Edward S. Hu[1,2] **Kwangjun Ahn**[1] **Qinghua Liu**[1] **Haoran Xu**[1,3] **Manan Tomar**[1]
Ada Langford[1] **Jayden Teoh**[1] **Bryon Xu**[1] **David Yan**[1] **Dinesh Jayaraman**[2]
Alex Lamb[1] **John Langford**[1]
[1]Microsoft Research [2]University of Pennsylvania [3]UT Austin

arXiv:2410.23506v3 [cs.LG] 16 Sep 2025

ABSTRACT

We introduce the "Belief State Transformer", a next-token predictor that takes both a prefix and suffix as inputs, with a novel objective of predicting both the next token for the prefix and the previous token for the suffix. The Belief State Transformer effectively learns to solve challenging problems that conventional forward-only transformers struggle with, in a domain-independent fashion. Key to this success is learning a compact belief state that captures all relevant information necessary for accurate predictions. Empirical ablations show that each component of the model is essential in difficult scenarios where standard Transformers fall short. For the task of story writing with known prefixes and suffixes, our approach outperforms the Fill-in-the-Middle method for reaching known goals and demonstrates improved performance even when the goals are unknown. Altogether, the Belief State Transformer enables more efficient goal-conditioned decoding, better test-time inference, and high-quality text representations on small scale problems. Website: https://edwhu.github.io/bst-website

1 INTRODUCTION

Transformer models (Vaswani et al., 2017) have created a revolution in language modeling (Achiam et al., 2023) with the capability to generate language with many emergent properties at large scale. Examining these models for flaws in the pursuit of further progress, it's notable that they struggle with planning-heavy problems (Bubeck et al., 2023; Momennejad et al., 2024). How can we modify the architecture, objectives, and algorithms to create a model more capable of reaching goals?

To make progress, we propose the new Belief State Transformer (BST) architecture and objective in Section 2. Informally, a belief state is a sufficient amount of information from the past to predict the outcomes of all experiments in the future, which can be expressed as either a distribution over underlying world states or a distribution over future outcomes. The Belief State Transformer is similar to a standard decoder-only Transformer (*e.g.*, GPT2), except that it has encodings that run both forward and backward. Both of these encodings are fed into output heads which predict not only the next token after the prefix *but also* the previous token before the suffix as shown in Figure 1.

In Section 3 we then study in depth how the Belief State Transformer performs on a known-hard problem, the star graph (Bachmann & Nagarajan, 2024) which is an elegantly simple sequential prediction problem known to confound next token prediction approaches. It's easy to show that transformers can represent star graph solutions using known results (*e.g.*, (Sanford et al., 2024; Frydenlund, 2024)), so the problem here is one of optimization. In particular, we discover that parity problems can be embedded within star graph problems, with parity known as difficult for gradient-based optimizers. Surprisingly, despite throwing away the backward encoder for inference, the BST solves even relatively difficult instances of star graphs with experiments detailed in Section 3.4. Analyzing this discovery, the BST benefits from extra gradients, enabling avoidance of the parity-by-gradient problem systematically. We also show that data augmentation approaches and ablations of the BST cannot solve the star graph problem systematically.

Building on this discovery, Section 4 proves this is a general phenomenon: ideal Belief State Transformers recover the full belief state in a compact representation for the output head. In contrast, a forward-only transformer and even modifications which predict every future token do not. This

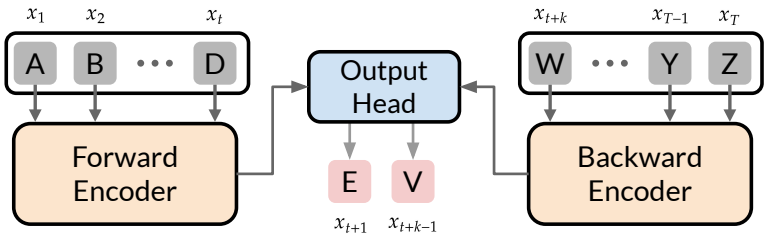


Figure 1: The Belief State Transformer has *two* encoders, one running forward and one backward with an output head for the next forward token and another for the previous backward token.

result implies that the Belief State Transformer learns maximal information from a sequence—there is no other objective/representation which pulls more relevant information into a compact belief state.

The Belief State Transformer creates new allowances which we further explore in Section 5. In particular, it's easy to specify a goal token (or tokens) explicitly to generate a goal-conditioned sequence. We compare this to the fill-in-the-middle approach (Bavarian et al., 2022), finding that the Belief State Transformer succeeds more effectively on the Tinystories dataset. The Belief State approach also enables inference planning because rollouts can occur with the Next head and a semi-independent evaluation of rollout quality can be enabled with the Prev head. Going further, we can use the Belief State Transformer as an embedder, with the relevant embeddings significantly superior to other transformer-based approaches.

Altogether, we show that the Belief State Transformer extracts more information (in theory and in practice) from a set of sequences, enabling Transformer models to perform well in new regimes. Performance at larger scale is of course an important question for further consideration.

2 The Belief State Transformer

This section introduces the Belief State Transformer. We start by introducing the architecture and the training objective then discuss how to utilize the model for inference.

2.1 Architecture and Objective

Let $x_{1:T}$ be shorthand for the sequence $x_1, ..., x_T$. First, we set up the following networks:

$$\begin{array}{lll} \text{Forward encoder} & \mathsf{F}(x_{1:t}) & \triangleright \text{ Encodes prefix} \\ \text{Backward encoder} & \mathsf{B}(x_{t+k:T}) & \triangleright \text{ Encodes suffix} \\ \text{Next decoder} & \hat{x}_{t+1} \sim \mathsf{T}_n(\cdot \mid \mathsf{F}(x_{1:t}), \mathsf{B}(x_{t+k:T})) & \triangleright \text{ Predicts next token} \\ \text{Prev decoder} & \hat{x}'_{t+k-1} \sim \mathsf{T}_p(\cdot \mid \mathsf{F}(x_{1:t}), \mathsf{B}(x_{t+k:T})) & \triangleright \text{ Predicts previous token} \end{array} \tag{1}$$

The forward encoder aggregates the prefix into a latent $\mathsf{F}(x_{1:t})$, and the backward encoder aggregates the suffix into a latent $\mathsf{B}(x_{t+k:T})$. We use GPT2-style encoders throughout our experiments, including baselines. The output heads T_n and T_p then predict their respective tokens. In our experiments, the parameters of T_n and T_p are tied with only the last layer differing. See Figure 1 for an illustration.

The Belief State Transformer objective is the straightforward sum of the objectives of forward and backward Transformers conditioned on the prefix and suffix.

$$\mathbb{E}_{t, x_{1:t}, k \leq T-t} \left[\log \frac{1}{\mathsf{T}_n(x_{t+1} \mid \mathsf{F}(x_{1:t}), \mathsf{B}(x_{t+k:T}))} + \log \frac{1}{\mathsf{T}_p(x_{t+k-1} \mid \mathsf{F}(x_{1:t}), \mathsf{B}(x_{t+k:T}))} \right] \tag{2}$$

An obvious alternative (called "Fill in the Middle" (Bavarian et al., 2022)) when both a prefix and suffix are available is simply putting them together and then using a forward encoder. Information-theoretically, Fill in the Middle works, but we'll see that the Belief State Transformer has several advantages: it causes the system to coalesce a compact belief state which has many benefits explored here. This approach also extracts $O(T^2)$ gradients from sequences enabling a gradient based optimizer to solve new problems as discussed in the next section. See Appendix E for code and scaling rules.

Training on all prefix-suffix pairs is surprisingly efficient. First, we cache all forward $f_{0:T} = \{\forall i \in [0 : T] : \mathsf{F}(x_{1:i})\}$ and backwards $b_{1:T+1} = \{\forall i \in [1, T+1] : \mathsf{B}(x_{i:T})\}$ latents. Then the loss Equation (2) is computed over training examples (f_i, b_j) and their labels (x_{i+1}, x_{j-1}) for valid i, j with $j - i > 1$. We first compute the gradients of the output decoder, and then add them up to compute the gradients of the encoders. Since there are $O(T)$ gradients over the output head per position of the forward or backward encoders this optimization saves a large amount of memory and compute.

2.2 Belief State Inference

During inference time, the forward model $\mathsf{T}_n(\mathsf{F}(x_{1:t}), \mathsf{B}(\emptyset))$ is given a prefix $x_{1:t}$ and an empty suffix $\emptyset$. Autoregressive sampling (ARS) is always used, where we sample the next token $\hat{x}$ from the next token decoder, add it to the prefix, and repeat. Note that since $\mathsf{B}(\emptyset)$ can be precomputed, this approach requires no more parameters at inference time than a standard forward-only Transformer. Later in Section 5, we study more complex inference approaches.

3 Testing Planning Abilities with Star Graphs

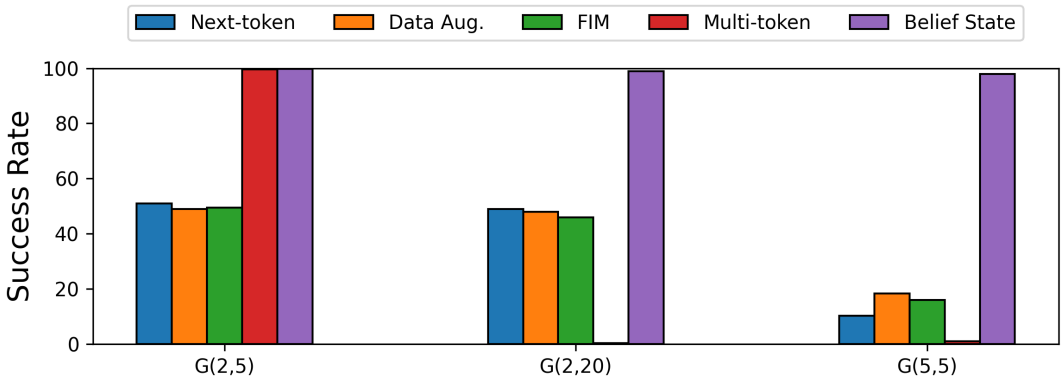


Figure 2: The Belief State Transformer outperforms baselines in all star graph navigation tasks.

Bachmann & Nagarajan (2024) propose the star graph problem as an elegantly simple yet challenging task for forward Transformers to solve. In Figure 2, we reproduce their results while adding a data augmentation baseline and the new Belief State Transformer results. Notably, the Belief State Transformer performs exceptionally well without relying on domain-specific adaptations. In the following sections, we explain the star graph problem, present a new theory to account for these results, provide a detailed discussion of our experiments, and ablate key design choices.

3.1 The Star Graph Problem

A star graph (depicted in Figure 3) $G(d, l)$ is a graph with d paths of length l emanating out from the start node. To construct a graph, nodes n_i are sampled uniformly from $\{1, \ldots, N\}$. A training example is formatted as a sequence containing the edge list $\mathcal{E}$, the start and end nodes, and a path of length l from start to end: $[\mathcal{E} \mid n_1, n_l \mid n_1, n_2, n_3, \ldots n_l]$. Despite its simplicity, modern next token prediction models fail to solve it.

This task captures a core challenge found in practical planning tasks like story writing, where creating a coherent narrative requires the author to keep the story's resolution and backstory in mind while progressing through each plot point.

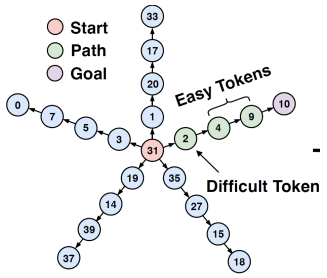


Figure 3: Illustration of the star graph problem from Bachmann & Nagarajan (2024).

3.2 WHY DO FORWARD-ONLY APPROACHES FAIL?

As shown by (Bachmann & Nagarajan, 2024, Appendix F.2) through extensive experiments, next-token predictors quickly learn a "*flawed cheat*" strategy: soon after training begins, forward-only transformers learn to arbitrarily select a neighbor of the current node since, aside from the start node, each node has only one outgoing edge. This flawed strategy leads the model to choose a neighbor of the start node without accounting for the designated goal during inference, limiting the test accuracy to $1/d$, where d is the number of neighbors of the start node. The key issue with this flawed cheat is that once the model learns it, finding the correct solution becomes exceedingly difficult. To better understand this challenge, we provide formal evidence of its difficulty for gradient optimization.

Theorem 1 (Informal)**.** *Once the "flawed cheat" strategy is perfected, learning the correct path is at least as difficult as learning full parity functions.*

A full proof is provided in Appendix B. The full parity problem is a notoriously hard problem for gradient-based optimizers (e.g., Shalev-Shwartz et al., 2017; Abbe & Sandon, 2023). In fact, it is conjectured that learning the full parity function requires exponentially many samples and computations in the input dimension (*e.g.*, Abbe & Boix-Adsera, 2022).

The analysis in this section indicates that once the flawed cheat is perfected, the limited supervision available for the task leads to significant intractability.

3.3 HOW DOES THE BELIEF STATE TRANSFORMER SUCCEED?

The Belief State Transformer is trained to predict the previous token for every suffix, which prevents the problem from collapsing into the parity problem described in Theorem 1. At a high level, for our approach to reduce to the parity problem, a large number of gradients must approach zero. However, the Belief State Transformer ensures that the information necessary to construct path suffixes (e.g., $n_{2:l}$) is present in the input to the output head. From this information, predicting n_2 is straightforward, allowing the model to solve the problem effectively.

This is an instance of a more general phenomenon: the Belief State Transformer naturally converges toward extracting a compact, complete belief state as discussed further in Section 4.1.

3.4 EXPERIMENTAL RESULTS ON STAR GRAPH

Here, we provide details on the Stargraph results in Figure 2. We run experiments on three types of graphs: $G(2, 5)$, $G(5, 5)$, $G(2, 20)$. In each experiment, we choose one graph topology and generate many example graph sequences, with all methods receiving the same amount of data and every baseline receiving at least as much computation as the Belief State Transformer uses. For evaluation, the models are conditioned on the edge list, start, goal, and current path, with the task of next node prediction: $p(n_i \mid \mathcal{E}, n_1, n_l, n_{1:i-1})$. We report the path accuracy, which is the percentage of correct path generations during the test time, over 10,000 evaluation graphs.

Empty suffix at inference. The Belief State Transformer trains an additional encoder B for suffixes. However, as discussed in Section 2.2, we remove the dependency on B during inference time by pre-computing the backward latent $b_\emptyset = \mathsf{B}(\emptyset)$ with the end token and proceed with auto-regressive sampling of the Next decoder $\mathsf{T}_n(\cdot \mid \mathsf{F}(\mathcal{E}, n_1, n_T, n_1, \ldots n_i), b_\emptyset)$. The model is still able to produce goal-conditioned behavior since the goal node n_T is present in the prefix input to the forward encoder.

Baselines. We select baselines that can be applied to a broad class of problems.

- **Forward-only next-token prediction:** This baseline follows the conventional strategy of training a Transformer with the next token prediction objective and teacher forcing, *i.e.*, the model is trained by feeding the correct previous token as input.
- **Data augmentation:** A common strategy to improve performance is to employ some form of data augmentation, although this requires some domain expertise to perform (Lee et al., 2024). This baseline augments the training data by replacing the goal with subgoals to potentially improve the learning of goal-conditioned behaviors. Specifically, the goal node, usually the terminal node in the path, is replaced with an intermediate node in the path. Then, a Transformer is trained with the next-token objective on this augmented dataset.